

Approximate Memristive In-Memory Hamming Distance Circuit

MOHAMMAD M. A. TAHA and CHRISTOF TEUSCHER, Portland State University

Hamming Distance (HD) is a popular similarity measure that is used widely in pattern matching applications, DNA sequencing, and binary error-correcting codes. In this article, we extend our previous work to prove that our HD circuit is scalable, tolerant to memristor model variability, and tolerant to device-to-device variation. We showed that the operation of our circuit under non-ideal fabrication conditions changes slightly, decreasing the correct classification rates for the MNIST handwritten digits dataset by $<1\%$. Our circuit's operation is independent of the memristor model used, as long as the model allows a reverse current. Because we leverage in-memory parallel computing, our circuit is $n\times$ faster than other HD circuits, where n is the number of HDs to be computed, and it consumes $\approx 100\times - 1,000\times$ less power compared to other memristive and CMOS HD circuits. Used in a full HD *Associative Content Addressable Memory* (ACAM), the proposed HD circuit consumes only 2.2% of the total system power. Our state-of-the-art, low-power, and fast HD circuit is relevant for a wide range of applications.

CCS Concepts: • **Hardware** → **Emerging technologies**;

Additional Key Words and Phrases: Hamming distance, memristor, memristive crossbar, in-memory computation, approximate computing

ACM Reference format:

Mohammad M. A. Taha and Christof Teuscher. 2020. Approximate Memristive In-Memory Hamming Distance Circuit. *J. Emerg. Technol. Comput. Syst.* 16, 2, Article 18 (March 2020), 14 pages.
<https://doi.org/10.1145/3371391>

1 INTRODUCTION

The *Hamming Distance* (HD) is a measure for the total number of positions where two bitstreams of equal length differ. It is a popular measure in information theory [6], error correction [20], and DNA sequence matching [11, 29]. Current HD circuits suffer from high power consumption and/or high latency due to the need for data transfer between the memory and the HD computing module. Memristors, which were first fabricated in 2008 by HP labs [21], can be used to lower the high power consumption and the high latency in circuits. The memristor is a two-terminal passive device that was first postulated by L. Chua in 1971; functionally, it is capable of storing a value that is represented in its resistance [4] and it thus provides an elegant means to combine

This work was supported by C-BRIC, one of six centers in JUMP, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

Authors' addresses: M. M. A. Taha and C. Teuscher, Electrical and Computer Engineering Department, Portland State University, 1900 SW 4th Avenue, Suite 160, Portland, OR, 97201; emails: {motaha, teuscher}@pdx.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2020 Association for Computing Machinery.

1550-4832/2020/03-ART18 \$15.00

<https://doi.org/10.1145/3371391>

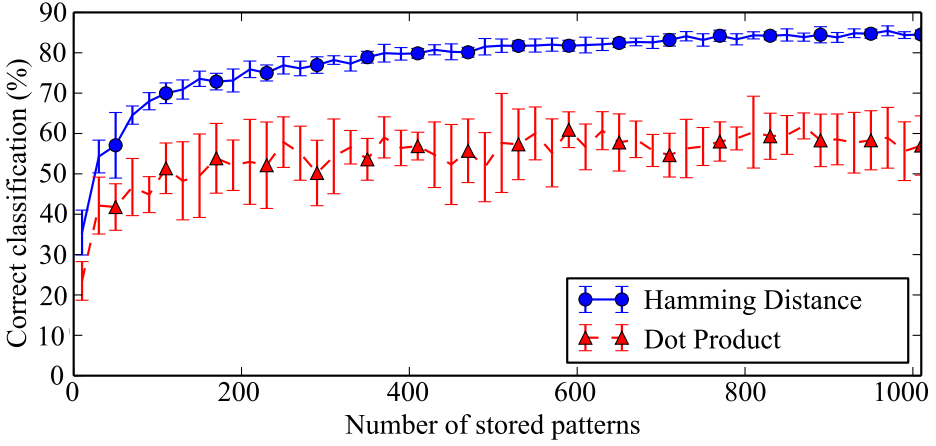


Fig. 1. MNIST software classification rates for the DP and HD distance metrics as a function of the number of stored patterns. The data was averaged over 10 runs.

both data storage and information processing. A conventional topology for memristive circuits is a nanowire crossbar, also known as *Resistive Crossbar Network* (RCN), where the memristors are located at the intersections between the horizontal and vertical nanowires (see Figure 7). RCNs can be integrated with high density and can work seamlessly together with traditional state-of-the-art CMOS circuits.

An RCN computes the *Degree Of Match* (DOM) between an input pattern and the stored patterns. The type of DOM achieved depends on the circuit topology. As a result, RCNs have gained popularity in the area of pattern matching and neuromorphic applications [5, 15]. Often, the achieved DOM is the *Dot Product* (DP), which may be achieved by adding a *Virtual Ground* (VGND) module at the end of each column to eliminate sneak path currents [24]. However, the RCN can be used to compute other DOMs, such as the HD [28]. These DOMs are sometimes more accurate in determining the best matching pattern. We previously compared the correct classification rates obtained when using the HD versus the DP on the MNIST dataset. We started by binarizing the dataset, then selecting random patterns to store, and testing them with the patterns from the testing dataset. The results showed that using the HD instead of the DP increases the classification rate on the MNIST dataset [26] from 57% to 85% (Figure 1). The reason for this difference in accuracy is as follows: when using the DP, if the input pattern has a “0” in a row, then that row will not contribute to the measured output value. In other words, a match between two “0”s and a mismatch between a “0” and a “1” results in the same output. The HD, on the other hand, is the count of rows (for example, x and y), where the corresponding bits do not match, regardless of the values of the bits; see Equation (1).

$$D_H(x, y) = |\{i : x_i \neq y_i\}| \quad (1)$$

In this article, we are extending our previous research [23] to test its operation on different pattern sizes, with different memristor models, under non-ideal fabrication conditions, and with datasets that have different sparsity levels. We also evaluated our circuit in a full memristive and CMOS system setup and determined its contribution to the overall power consumption and performance.

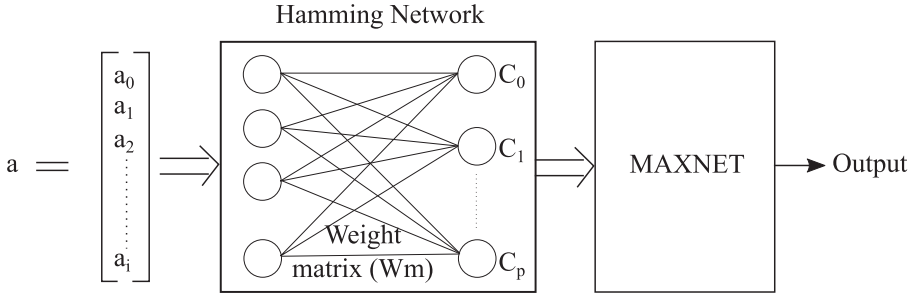


Fig. 2. Block diagram of a minimum HD classifier using the Hamming network and the MAXNET. The number of classes in this network is p . The MAXNET is a recurrent network that has both inhibitory and excitatory connections. It is considered a *Winner-Take-All* (WTA) system as it suppresses all the Hamming network outputs but the largest one. The output corresponds to the stored pattern that most closely matches the input. For more details on the MAXNET, see Ref. [8].

2 RELATED WORK

2.1 Hamming Distance Content Addressable Memories (CAMs)

Architectures that calculate the HD have been used before to build Content Addressable Memories (CAMs). For example, Yusuke et al. [27] designed a hierarchical multi-chip architecture of fully parallel HD associative memory units. Although their multi-chip structure achieved good capacity and scalability, they implemented the HD module as a separate block, which increased the latency and resulted in a power consumption of 51.3 mW for a 64×32 (word size $\times$ number of words) CAM. Mattausch et al. [14] designed an associative memory architecture that has low area requirements. However, their design still suffered from high latency and high power consumption of 260 mW. Rahimi et al. [16] proposed an *Approximate Associative Memristive Memory* (A^2M^2) for energy-efficient GPUs, which depended on modifying the *Ternary Content Addressable Memory* (TCAM) cell design of Li et al. [13] by replacing some of the CMOS transistors with memristive devices. Although this lowered the power consumption of the original TCAM, the A^2M^2 design still featured some CMOS transistors, which limited the area-saving potential of using memristive devices. Their design also had a higher operating voltage than the approach in this work; our lower operating voltage should result in additional power savings. Unfortunately, a direct power comparison is impossible as Rahimi et al. only provide energy data for the entire system, and not separately for the A^2M^2 .

2.2 Hamming Distance Circuits

The Hamming network and MAXNET is a two-layer *Artificial Neural Network* (ANN) classifier of bipolar vectors [8]. This network is capable of identifying the pattern, from its stored ones, that most closely matches the input according to the HD metric. The number of input a and output C neurons is equal to the number of bits per pattern and the number of classes, respectively; see Figure 2.

As shown in Figure 2, the vector/pattern a consisting of i bits is presented to the Hamming network, and the neuron firing the most at the MAXNET corresponds to the pattern that has the lowest HD to the input, labeled as “Output.” The weight matrix of the Hamming network is created by encoding the class vectors. Let $s^{(C_1)}$ and $s^{(C_2)}$ be vectors/patterns relating to classes 1 and 2, respectively. The HD can then be computed by:

$$a^t s^{(x)} = \left(i - HD(a, s^{(x)}) \right) - HD(a, s^{(x)}), \quad (2)$$

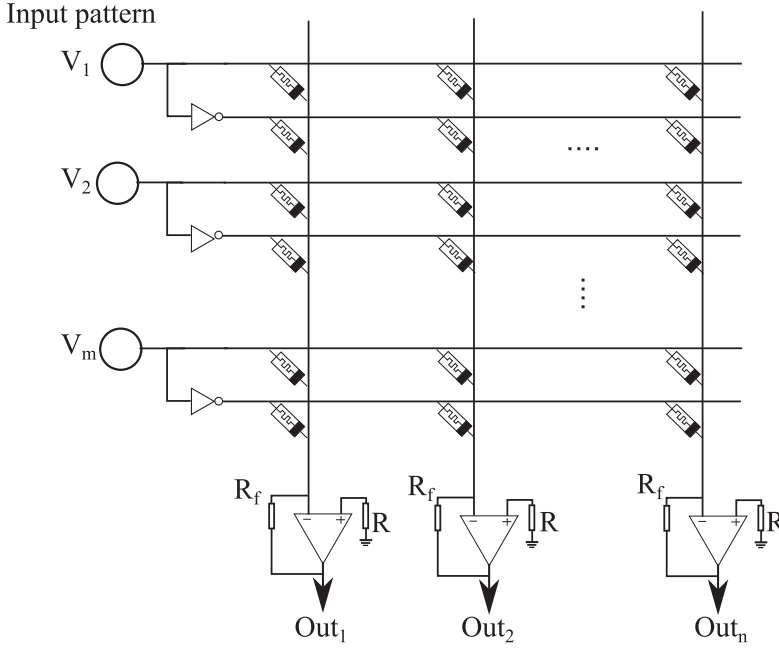


Fig. 3. Hamming distance circuit proposed by Zhu [28]. Redrawn from Ref. [28].

which can be written as:

$$\frac{1}{2} a^t s^{(x)} = \frac{i}{2} - HD(a, s^{(x)}), \quad (3)$$

where $x = \{0, 1, 2, \dots, p\}$, and a , s , and i are the input pattern, stored patterns, and the number of bits per pattern, respectively. The network's weight matrix W_m can be written as:

$$W_m = \frac{1}{2} \begin{bmatrix} s_0^{(C_0)} & s_1^{(C_0)} & s_2^{(C_0)} & \dots & s_i^{(C_0)} \\ s_0^{(C_1)} & s_1^{(C_1)} & s_2^{(C_1)} & \dots & s_i^{(C_1)} \\ s_0^{(C_2)} & s_1^{(C_2)} & s_2^{(C_2)} & \dots & s_i^{(C_2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ s_0^{(C_p)} & s_1^{(C_p)} & s_2^{(C_p)} & \dots & s_i^{(C_p)} \end{bmatrix},$$

where the $\frac{1}{2}$ is a scaling factor. The output of each neuron of the Hamming network is:

$$f(output) = \frac{1}{i} \left(\frac{1}{2} a^t s^{(x)} + \frac{i}{2} \right), \text{ for } x = 1, 2, \dots, p, \quad (4)$$

where $\frac{i}{2}$ is the network's bias, and $a^t s^{(x)}$ is the DP between a^t and all the bipolar stored patterns in W_m [8]. Zhu et al. [28] proposed a hybrid Hamming network circuit based on CMOS and memristors. Since they needed to store each vector in a bipolar form, they used two memristors per bit along with NOT gates at the inputs, as shown in Figure 3. This, along with the VGND modules, increased both their design size and power consumption.

Cassuto and Crammer [3] developed a framework to calculate the Hamming similarity between vectors stored in a resistive memory. Their approach is not designed to calculate the HD between an input and the stored patterns but only between the stored patterns themselves, which makes it inefficient for pattern/image recognition applications. Ge et al. [7] proposed an HD comparator

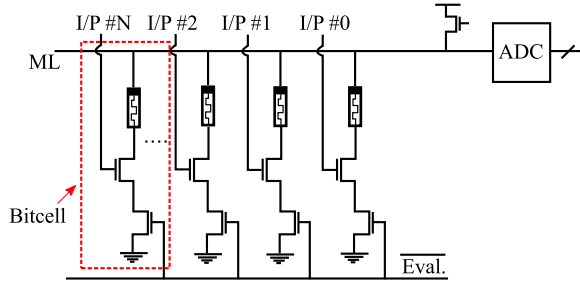


Fig. 4. RASSA's Sub-Word. Redrawn from Kaplan et al. [11].

using a unipolar memristor array. They used the ON/OFF switching characteristics of the memristive array to compute the HD between two vectors presented as voltages. Their design is also inefficient in pattern matching applications because (1) the patterns have to be brought from a different memory module, which increases the latency and power consumption, and (2) their design can only compute the HD between two vectors at a time. In order to compute the HD between an input and several patterns, they either can do it serially, which is time-consuming, or have multiple circuits/memristive arrays, which, in turn, increases the power consumption. Kaplan et al. [11] proposed a resistive pre-alignment accelerator for approximate DNA long read mapping “RASSA” that computes the HD between DNA strands. Their design featured several word rows, each row consisting of multiple 2T1R bitcells connected by a *Match Line* (ML) to an *Analog to Digital Converter* (ADC), see Figure 4. Since their HD calculation depends on the ML discharge amount, they used a single 4-bit ADC per up to 60 bitcells to sense the ML accurately and reduce the power consumption. Their design consumed 1.8 mW for a single comparison cycle for a 960 bits pattern/strand. An issue with their design is that it can only detect a mismatch between an input “1” and a stored “0,” but not the opposite. This is acceptable when using one-hot encoding for DNA base-pair strands to detect a single mismatch. However, it will be inefficient in other applications that require the detection of a mismatch between an input “0” and a stored “1.”

Zokaee et al. [29] adopted a similar design to Ge et al. [7] to accelerate the FM-Index in DNA short read alignment. Although their design improves the read’s throughput per Watt per mm^2 by $13\times$ compared to the state-of-the-art, it requires a 4×4 ReRAM array of 16 memristors to compute the HD between 2-base pairs (4 bits), which increases the area and power consumption.

Huang et al. [9] proposed an FPGA architecture for on-board corner detection and matching. One of their sub-systems computes the HD between two binary patterns. Their HD circuit consists of a multi-input XOR gate and an adder tree, as shown in Figure 5, where the XOR finds the difference between two patterns, and the adder tree counts the number of differences. Since this design computes the HD between two patterns only, they used n circuits to parallelize the HD computation between an input and n stored patterns. They did not report any power numbers, so we implemented their HD circuit on the Xilinx XC7K325T FPGA. Our results showed that 0.425 W of power is required to compute the HD between one input pattern and 10 stored 256-bit patterns.

Figure 6 shows an accuracy and power consumption comparison of the previously mentioned HD circuits.

3 METHODOLOGY

Figure 7 shows a basic RCN consisting of horizontal and vertical nanowires with memristors located at the wires’ intersections. The binary patterns are stored in the columns, one pattern per column, with the memristor set to R_{OFF} (high resistance) to represent binary “0,” and R_{ON} (low

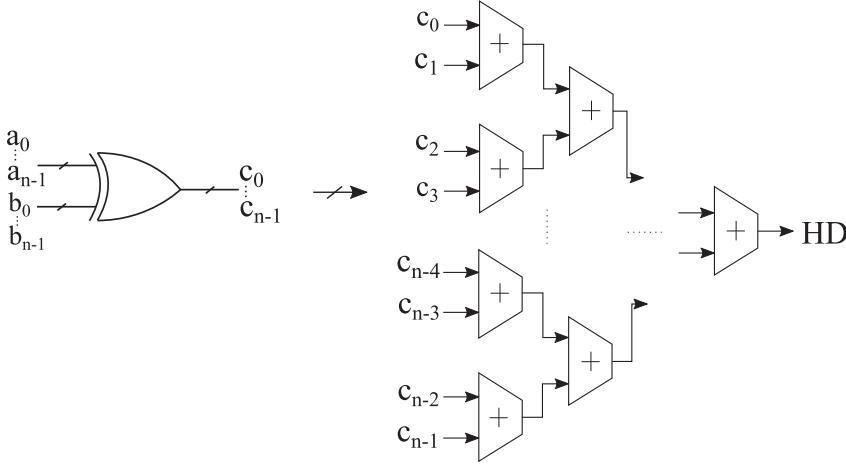


Fig. 5. Hamming distance circuit as implemented in an FPGA [9].

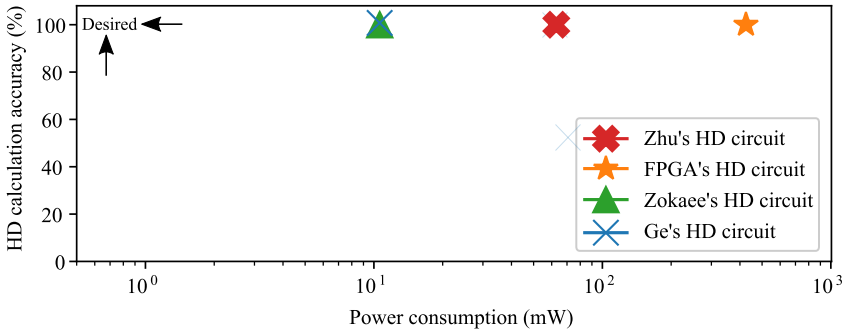


Fig. 6. An accuracy versus power comparison of related work when computing the HDs for 10 256-bit patterns. Ideal circuits would be located in the top left corner.

resistance) to represent binary “1.” The input/test pattern is introduced to the crossbar through V_1 to V_m , where a read voltage V_{read} represents binary “1” and 0 V represents binary “0.”

When the input pattern is presented as voltages to the crossbar rows, the current flowing through the memristor with conductance g_{ij} , where i and j denote the row and column number, respectively, is $V_i \times g_{ij}$. Therefore, the total current through column j is $\sum_i V_i \times g_{ij}$ when the crossbar columns are grounded. The total current of each column represents the correlation or DOM between the input and the pattern stored in that column; thus, the best matching pattern to the input is the pattern corresponding to the highest correlation magnitude $\sum_i V_i \times g_{ij}$ [18, 24]. Choosing the highest output between two crossbar columns can be done using the WTA circuit shown in Figure 8. This WTA can be extended to have multiple inputs depending on the number of crossbar columns, 1 WTA input per crossbar column.

3.1 Hamming Distance Evaluation

A common architecture to compute the DP in-memory is the RCN with VGND modules terminating its columns [24]. The VGND modules are required to terminate the crossbar columns to eliminate the traditionally undesired sneak paths currents. When computing the DP, the mismatch between a stored “1” and an input “0” does not affect the output. In the HD computation, we

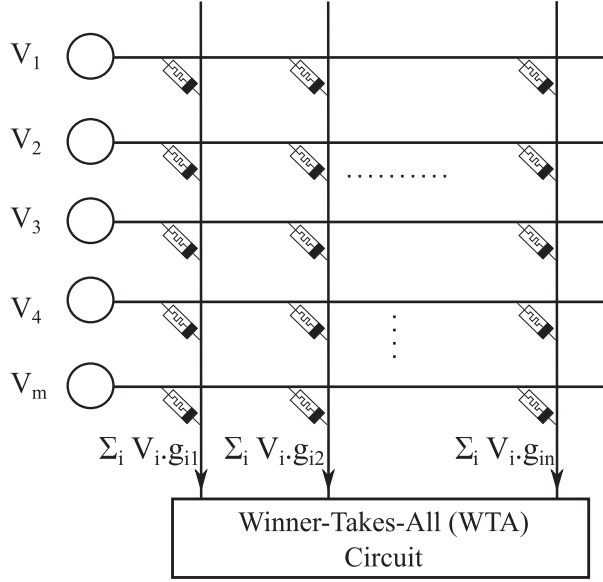


Fig. 7. An RCN where the memristors are located at the intersection of the horizontal and vertical nanowires. RCNs are used for storing patterns and evaluating the correlation between an input and the stored patterns. The patterns are stored in the columns of the RCN, and the binary input pattern is applied to the RCN rows in the form of voltages with V_{read} or 0 V, representing binaries “1” and “0,” respectively.

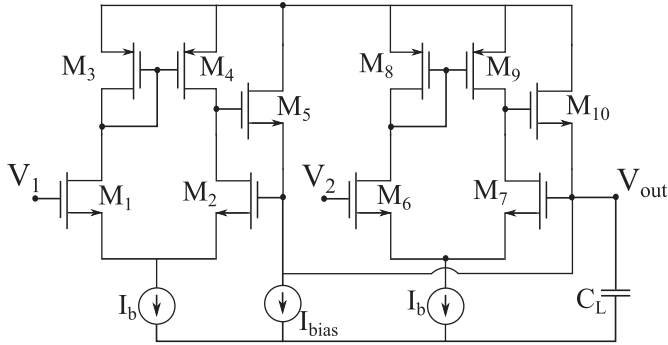


Fig. 8. Two-input analog MAX/WTA circuit. Redrawn from Ref. [2].

require this mismatch to influence the total column's current. Practically, this can be achieved by inducing a voltage drop at the end of each column rather than using a VGND module. When there is a positive voltage drop on each column, that voltage produces a current back through the memristive devices corresponding to rows where a “0” is input. If these rows were storing a “1,” then more current would leave through the device than if the row was storing a “0,” (because a stored “1” is represented by low resistance R_{ON} while “0” is R_{OFF} , which is high resistance); this is illustrated in Figure 9. The result is that the voltage of a column will be lower if there are more mismatches. Thus, the column with the highest output voltage will be the column corresponding to the best matching stored pattern according to an approximate HD calculation. This approximation is due to the difference between the currents in the different mismatch cases. That is, a mismatch between an input “0” to a stored “1” does not have the same weight as a

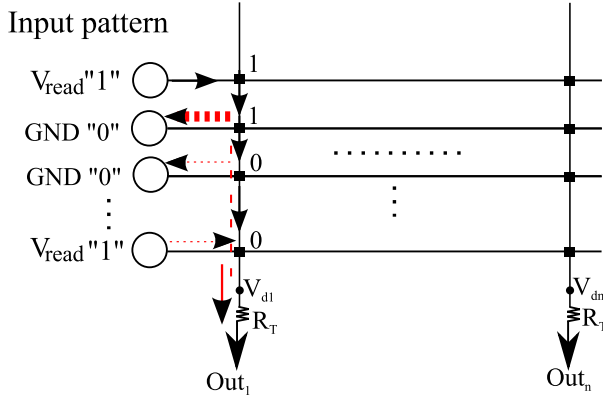


Fig. 9. This illustration demonstrates how sneak path currents are leveraged to compute the HD. Only a single column's currents are shown, but each column is independent in this circuit, so the same principle applies to the other columns. When the input pattern is presented to the RCN, current flows both into and out of the RCN columns. When there is a voltage drop at the end of the column V_{d1} , currents will flow from the column to the GND nodes provided by "0"s in the input pattern. The currents flowing through a memristor storing a "1" (thick dotted arrow) are larger than currents flowing through a "0" (thin dotted arrow) because a "0" represents high resistance and a "1" represents low resistance. This is beneficial for computing the HD, as currents through a "0" device to the ground (which represents a stored "0" with an input "0") will be smaller than currents through a "1" device (stored "1" to an input "0"). The termination resistance R_T can be omitted when "0" inputs are grounded, and as long as the circuit connected to the RCN induces a voltage drop greater than 0V on the columns. Redrawn from Ref. [23].

mismatch between an input "1" to a stored "0." In addition, there is still some lost current, sunk to the GND node, when there is a match between a "0" and a "0." However, because of the memristive range, this current is negligible when compared to the sunk current due to mismatches.

4 RESULTS AND DISCUSSION

The experiments here were simulated using the Xyce open source, SPICE-compatible, high-performance analog circuit simulator [17]. They were performed on a binary dataset that we generated using Python's NumPy random generator. The patterns used had 40% to 55% density levels unless otherwise noted. We chose to generate our dataset to be able to scale and change it to meet our experiments' test criteria. It is worth noting that the resistance of the nanowires was taken into consideration when performing these experiments. This resistance was taken from Ref. [10], and was set to 30 K Ω for a 1kb crossbar array.

4.1 Scalability

We tested our circuit by increasing both the number of bits per pattern and the number of patterns per crossbar.

4.1.1 Increasing the Number of Bits Per Pattern. As a benchmark, we performed the same experiment, using the same patterns, with a software HD computing function. Figure 10 shows our HD circuit's accuracy when increasing the number of bits.

From Figure 10, one can see that the average HD calculation accuracy is $\approx 95\%$. However, for the patterns with 6 to 10 bits, the accuracy is $\approx 79\%$. This is due to the weight of the mismatch between an input "0" and a stored "1" being higher than the mismatch of "1" and "0." For example, take the case when a stored pattern has an HD of 2, to the input, with two mismatches of input "1" to a

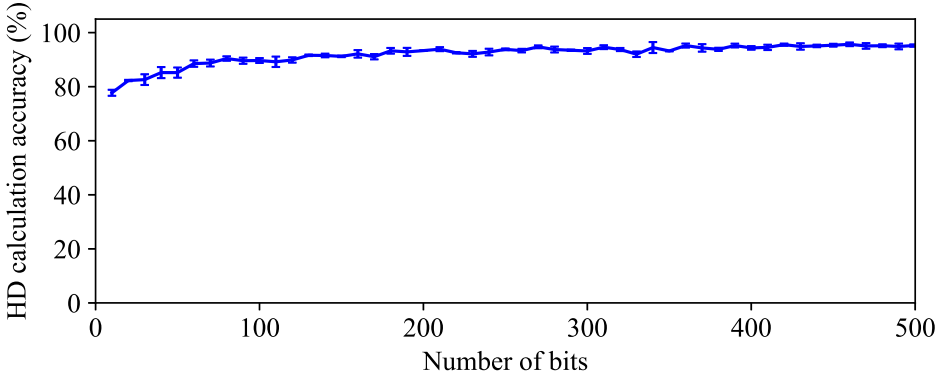


Fig. 10. The HD calculation accuracy when increasing the number of bits per stored pattern while having a fixed number of stored patterns. The number of stored patterns was fixed to 10.

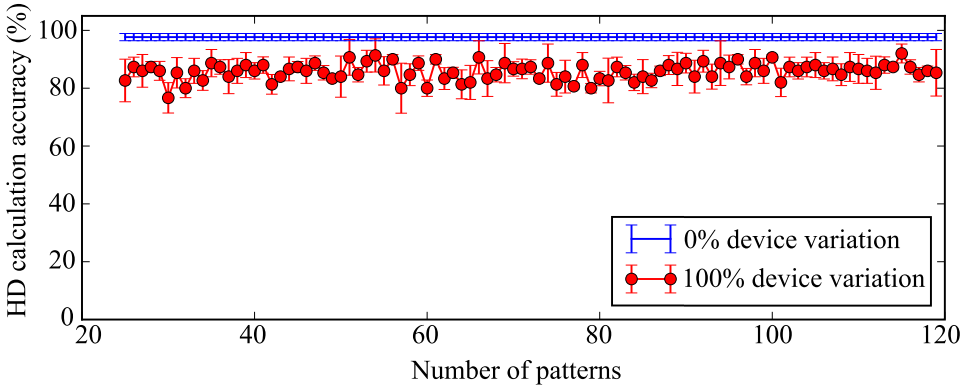


Fig. 11. The HD calculation accuracy when increasing the number of stored patterns while keeping the number of bits fixed to 256.

stored “0,” and another pattern having an HD of 1 with a mismatch of “0” to “1.” The HD circuit will output a higher voltage for the first pattern, although the closest should be the second one. Given the fast, parallel, and low power computation of our circuit, this decrease in performance can still suit applications that can sacrifice some accuracy to gain speed and save power.

4.1.2 Increasing the Number of Patterns. In order to see the effect of increasing the stored patterns on our circuit’s performance, we stored and retained a constant 20 patterns while introducing new patterns. When computing the performance, we only compared the outputs from the constant 20 patterns to see whether their outputs would be affected by the increase of the number of stored patterns. From Figure 11, one can see that increasing the number of patterns did not affect our circuit’s performance; it remained constant at $\approx 96\%$.

4.2 Sparsity

The goal of the following experiments was to test the effect of the dataset sparsity levels on the performance of our HD circuit. We generated binary datasets using Python’s NumPy random generator with different “1” density percentages. Figure 12(a) shows the HD calculation accuracy at each density level. Figure 12(b) shows the average HD calculation accuracy when the dataset has a

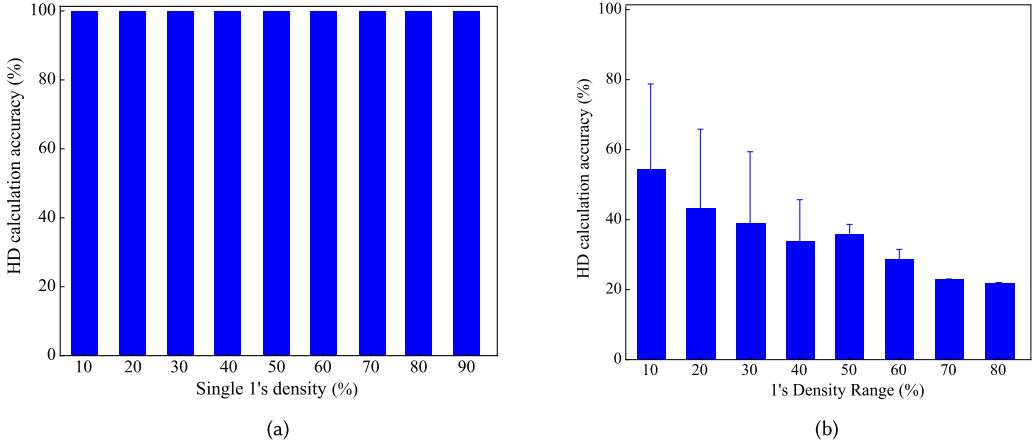


Fig. 12. (a) HD calculation accuracy obtained when having a single constant sparsity/density for all the patterns. (b) HD calculation accuracy obtained when having a range of sparsity/density levels for the patterns. The number of bits used was 256.

range of “1” density, e.g., 10% range means that the dataset patterns have different densities within 10% of each other, and so on. One can see that our circuit’s performance decreases when the range of densities increases. This is because, as mentioned before, the mismatch between “0” to “1” has a higher weight than the mismatch between “1” to “0,” and a higher density range increases the probability of having more “0” to “1” mismatches than lower density ranges. That said, our HD circuit achieved $\approx 96\%$ on single densities, and, as can be seen in Table 2, it achieved classification rates within 1% of the rates obtained by other HD circuits on the MNIST dataset.

4.3 Robustness to Device Variability

Memristors are prone to variations because their resistance change is determined by the migration of oxygen vacancies [19]. We performed experiments to test the operation of our HD circuit under device variations. We used the Yang model [25] in this experiment as its device-to-device variations were reported in Ref. [19]. The Yang model is a metal oxide WO_x memristor in which the electric field causes the oxygen vacancies to move and thus modulates the resistance and results in the characteristic conductance hysteresis. This memristor can be modeled by:

$$I = w(\gamma \sinh(\delta V)) + (1 - w)(\alpha(1 - e^{-\beta V})) \quad (5)$$

$$\frac{dw}{dt} = \eta_1 \sinh(\eta_2 V) F(w, V), \quad (6)$$

where $\gamma, \delta, \alpha, \beta, \eta_1$, and η_2 are the material parameters, w is the state variable between 0 and 1, and $F(w, V)$ is window function to account for the oxygen vacancies nonlinear change as described in Ref. [25].

Having device variations means that the resistances and current flow through each memristor will be different. We tested our circuit with variations according to the numbers given in Ref. [19]. Figure 11 shows the HD calculation accuracy with 100% device-to-device variation when increasing the number of stored patterns. The results were averaged over 10 iterations. One can see that the HD calculation accuracy is lowered to an average of 92%.

Table 1. Proposed HD Circuit Performance When Tested with Four Different Memristor Models

Memristor model	Yang and Lu [25]	Berdan et al. [31]	Merrikh-Bayat et al. [30]	Jo et al. [32]
HD classification accuracy	94.3%	93.8%	93.9%	93.2%

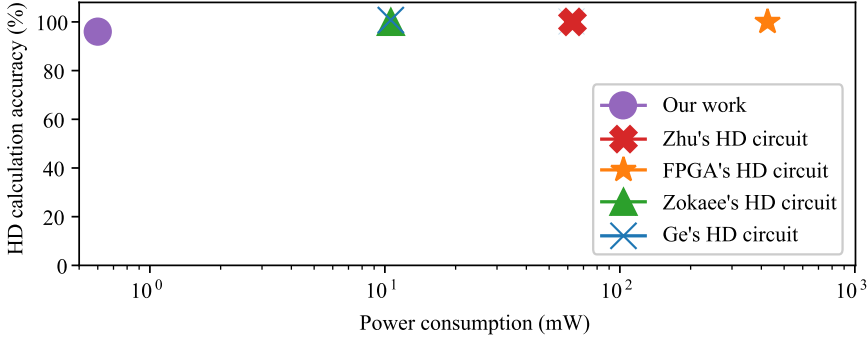


Fig. 13. An accuracy versus power consumption comparison of related work when computing the HDs for 10 256-bit patterns. Note that the power of the WTA circuit is not included since the other circuits were simulated without one.

4.4 Memristor Model Variability

In this experiment, we tested our HD circuit operation when using different memristor models. Table 1 shows that the performance of the circuit changed by <1%. The characteristics of the memristor models tested were as follows:

- have R_{OFF} to R_{ON} ratios ranging from $3\times$ to $18\times$;
- are built from different materials such as Titanium Ti , Copper Cu , Silver Ag , and Silver Chalcogenide Ge_2Se_3 and Ag , which gives them different I-V characteristics (symmetrical and asymmetrical); and
- allow reverse current to flow normally through the memristor without suppressing it. So, for example, the Lehtonen model in Ref. [12] will not be suitable for our circuit because it suppresses the reverse current. This suppression will affect the detection of a “0” to “1” mismatch and only detect a “1” to “0” mismatch.

4.5 Comparison to Related Work

Figure 13 shows the accuracy and power consumption of our and other HD circuits. Although our circuit achieves $\approx 96\%$ accuracy only, it consumes less power. This makes our circuit beneficial for low power applications where one can sacrifice some accuracy. The work of Kaplan et al. is not depicted in Figure 13 because their reported power number includes other circuitry, which we were not able to reproduce. However, it is worth noting that their HD circuit bitcell consists of 2T1R while ours is only 1R. Furthermore, our circuit, unlike theirs, can detect a mismatch between an input “0” and a stored “1.”

4.5.1 Datasets and Full System. In this section, we tested our circuit on the MNIST handwritten digits and benchmarked it against the other HD computing circuits/schemes. Table 2 shows the correct classification rates obtained over 100 stored patterns. One can see that our circuit’s obtained rates with and without device variations are within 1% of the results obtained by the other HD

Table 2. Classification Rates Obtained for Our HD Circuit, an HD Python Software Implementation, and Other HD Circuits from the Literature

HD circuit type	Python	Proposed 0% device variation	Proposed 100% device variation	Zhu et al. [28]	Zokaee et al. [29]	Ge et al. [7]	FPGA [9]
Classification rates	72.7%	72.5%	71.2%	72.7%	72.7%	72.7%	72.7%
HD to rest of ACAM system's power ratio	N/A	0.02	0.024	3.3	7.5	7.5	228 [†]

[†]This percentage is calculated, not simulated, because we could not integrate the FPGA with the rest of the simulated full ACAM system design.

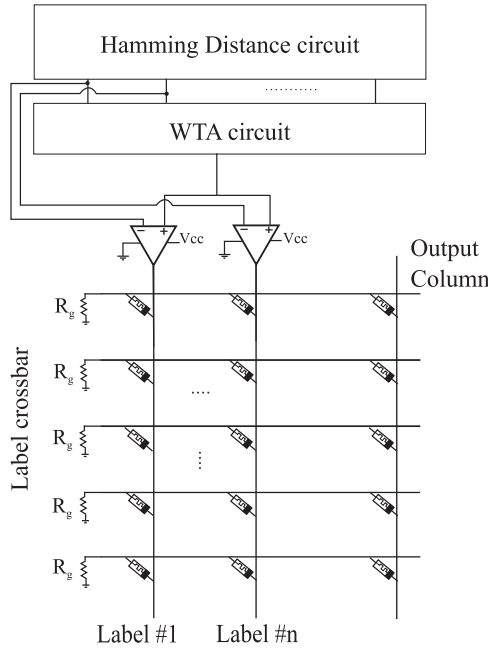


Fig. 14. The full ACAM system. The various HD circuits tested were placed in the “Hamming Distance circuit” box. The output of the “WTA circuit” and the “Hamming Distance circuit” is input to analog comparators, which output a one-hot code, where the location of the “1” represents the location of the winning column. These outputs are connected to the “Label Crossbar” to choose the winning label. The “Label Crossbar” performs a NOT operation on the memristors located in the winning column and writes the output to the “Output Column.” For more details on how the NOT operation is performed, see Ref. [1]. As a result of this NOT operation, the inverse of the labels corresponding to the stored images are written to the label crossbar before the inference phase.

circuits/schemes. This confirms the operation of our circuit and proves that our HD circuit can tolerate large device-to-device variation.

To determine the impact of each circuit on the full system, we also tested our HD circuit versus the other HD circuits in a full HD *Associative Content Addressable Memory* (ACAM), which we proposed in Ref. [22]. Figure 14 shows the ACAM system, which takes an input pattern and outputs

a label. Table 2 shows the power ratio of each HD circuit to the rest of the system. One can see that our circuit had the smallest power ratio, thus lowering the full system's power consumption while leaving room for decreasing the power consumption more by improving the rest of the system.

5 CONCLUSION

The presented work is an extension of previous research [23] conducted by the authors. Here, we showed that increasing the size of our circuit either in rows or columns does not affect its performance and that our circuit is tolerant to device-to-device variations. When testing it with 100% device variations and by considering the nanowire resistance, its performance only decreased from $\approx 96\%$ to $\approx 92\%$. Our circuit's operation is also independent of the memristor model used as long as the model allows reverse current. Compared to other HD circuits, ours is $n \times$ faster, where n is the number of calculated HDs. Our circuit consumes $\approx 100 \times - 1,000 \times$ less power because it relies on memristors and sneak path currents to compute the HD, which eliminated the need for the NOT gates, VGND modules, and the supporting CMOS circuitry. Although the performance of our HD circuit decreases when the "1"s density range in the stored patterns increases, the power consumption is still $\approx 100 \times - 1,000 \times$ lower compared to other HD circuits. Used in a full HD *Associative Content Addressable Memory* (ACAM), our HD circuit consumes only 2.2% of the total system power. Our work is most relevant for designing low-power applications that can sacrifice some accuracy.

REFERENCES

- [1] J. Borghetti, G. Snider, P. Kuekes, J. Yang, D. Stewart, and R. Williams. 2010. "Memristive" switches enable "stateful" logic operations via material implication. *Nature* 464, (2010), 873–876. DOI: <https://doi.org/10.1038/nature08940>
- [2] R. Carvajal, J. Ramirez-Angula, and J. Tombs. 2000. High-speed high-precision voltage-mode MIN/MAX circuits in CMOS technology. In *Proceedings of the 2000 IEEE International Symposium on Circuits and Systems (ISCAS)* 5 (2000), 13–16. DOI: <https://doi.org/10.1109/ISCAS.2000.857351>
- [3] Y. Cassuto and K. Crammer. 2015. In-memory Hamming similarity computation in resistive arrays. In *Proceedings of the 2015 IEEE International Symposium on Information Theory (ISIT)*. 819–823. DOI: <https://doi.org/10.1109/ISIT.2015.7282569>
- [4] L. Chua. 1971. Memristor-the missing circuit element. *IEEE Transactions on Circuit Theory* 18, 5 (1971), 507–519. DOI: <https://doi.org/10.1109/TCT.1971.1083337>
- [5] D. Fan, M. Sharad, A. Sengupta, and K. Roy. 2016. Hierarchical temporal memory based on spin-neurons and resistive memory for energy-efficient brain-inspired computing. *IEEE Transactions on Neural Networks and Learning Systems* 27, 9 (2016), 1907–1919. DOI: <https://doi.org/10.1109/TNNLS.2015.2462731>
- [6] G. Forney. 1966. Generalized minimum distance decoding. *IEEE Transactions on Information Theory* 12, 2 (1966), 125–131. DOI: <https://doi.org/10.1109/TIT.1966.1053873>
- [7] N. Ge, J. Yoon, M. Hu, E. Merced-Grafals, N. Davila, J. Strachan, Z. Li, H. Holder, Q. Xia, X. Zhou Williams, R. S., and J. Yang. 2017. An efficient analog Hamming distance comparator realized with a unipolar memristor array: A showcase of physical computing. *Scientific Reports* 7, Article 40135 (2017). DOI: <https://doi.org/10.1038/srep40135>
- [8] A. Gupta and Y. Singh. 2011. Analysis of Hamming network and MAXNET of neural network method in the string recognition. In *Proceedings of the 2011 International Conference on Communication Systems and Network Technologies (CSNT)*. 38–42. DOI: <https://doi.org/10.1109/CSNT.2011.15>
- [9] J. Huang, G. Zhou, X. Zhou, and R. Zhang. 2018. A new FPGA architecture of FAST and BRIEF algorithm for on-board corner detection and matching. *Sensors* 18, 4 (2018), 1014. DOI: <https://doi.org/10.3390/s18041014>
- [10] S. Jo, K. Kim, and W. Lu. 2009. High-density crossbar arrays based on a Si memristive system. *Nano Letters* 9, 2, (2009), 870–874. DOI: <https://doi.org/10.1021/nl8037689>
- [11] R. Kaplan, L. Yavits, and R. Ginosar. 2019. RASSA: Resistive pre-alignment accelerator for approximate DNA long read mapping. *IEEE Micro* 39, 4, (2019), 44–54. DOI: <https://doi.org/10.1109/MM.2018.2890253>
- [12] E. Lehtonen, J. Tissari, J. Poikonen, M. Laiho, and L. Koskinen. 2014. A cellular computing architecture for parallel memristive stateful logic. *Microelectronics Journal* 45, 11 (2014), 1438–1449. DOI: <https://doi.org/10.1016/j.mejo.2014.09.005>

- [13] J. Li, R. K. Montoye, M. Ishii, and L. Chang. 2014. 1 Mb 0.41 μm^2 2T-2R cell nonvolatile TCAM with two-bit encoding and clocked self-referenced sensing. *IEEE Journal of Solid-State Circuits* 49, 4 (2014), 896–907. DOI : <https://doi.org/10.1109/JSSC.2013.2292055>
- [14] H. Mattausch, T. Gyohten, Y. Soda, and T. Koide. 2002. Compact associative-memory architecture with fully parallel search capability for the minimum Hamming distance. *IEEE Journal of Solid-State Circuits* 37, 2 (2002), 218–227. DOI : <https://doi.org/10.1109/4.982428>
- [15] D. Querlioz, W. Zhao, P. Dollfus, J. Olivier, O. Bichler, and C. Gamrat. 2012. Bioinspired networks with nanoscale memristive devices that combine the unsupervised and supervised learning approaches. In *Proceedings of the 2012 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*. 203–210. DOI : <https://doi.org/10.1145/2765491.2765528>
- [16] A. Rahimi, A. Ghofrani, K. T. Cheng, L. Benini, and R. K. Gupta. 2015. Approximate associative memristive memory for energy-efficient GPUs. In *Proceedings of the 2015 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1497–1502. DOI : <https://doi.org/10.7873/DATE.2015.0579>
- [17] Sandia National Laboratories. 2016. Xyce Simulator. Retrieved from <https://xyce.sandia.gov>.
- [18] M. Sharad, D. Fan, and K. Roy. 2013. Ultra low power associative computing with spin neurons and resistive crossbar memory. In *Proceedings of the 50th ACM/EDA/IEEE Annual Design Automation Conference (DAC)*. 1–6. DOI : <https://doi.org/10.1145/2463209.2488866>
- [19] P. Sheridan, C. Du, and W. Lu. 2016. Feature extraction using memristor networks. *IEEE Transactions on Neural Networks and Learning Systems* 27, 11 (2016), 2327–2336. DOI : <https://doi.org/10.1109/TNNLS.2015.2482220>
- [20] A. M. Steane. 1996. Error correcting codes in quantum theory. *Physical Review Letters* 77, 5 (1996), 793–797. DOI : <https://doi.org/10.1103/PhysRevLett.77.793>
- [21] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams. 2008. The missing memristor found. *Nature* 453, (2008), 80–83. DOI : <https://doi.org/10.1038/nature06932>
- [22] M. M. A. Taha, W. Chavez, and C. Teuscher. 2017. Spatial and temporal probabilistic inference using a memristive associative memory. *International Journal of Unconventional Computing* 13, 2 (2017), 97–115.
- [23] M. M. A. Taha, W. Woods, and C. Teuscher. 2016. Approximate in-memory Hamming distance calculation with a memristive associative memory. In *Proceedings of the 2016 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*, 159–164.
- [24] W. Woods, M. M. A. Taha, D. Tran, J. Bürger, and C. Teuscher. 2015. Memristor panic—A survey of different device models in crossbar architectures. In *Proceedings of the 2015 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*. 106–111. DOI : <https://doi.org/10.1109/NANOARCH.2015.7180595>
- [25] Y. Yang and W. Lu. 2013. Nanoscale resistive switching devices: Mechanisms and modeling. *Nanoscale* 5, 21 (2013), 10076–10092. DOI : <https://doi.org/10.1039/C3NR03472K>
- [26] L. Yann, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE* 86, 11 (1998), 2278–2323. DOI : <https://doi.org/10.1109/5.726791>
- [27] O. Yusuke, I. Makoto, and A. Kunihiro. 2004. Hierarchical multi-chip architecture for high capacity scalability of fully parallel Hamming-distance associative memories. *IEICE Transactions on Electronics* 87, 11 (2004), 1847–1855.
- [28] X. Zhu, X. Yang, C. Wu, J. Wu, and X. Yi. 2013. Hamming network circuits based on CMOS/memristor hybrid design. *IEICE Electronics Express* 10, 12 (2013). DOI : <https://doi.org/10.1587/ele.10.20130404>
- [29] F. Zokaee, H. R. Zarandi, and L. Jiang. 2018. Aligner: A process-in-memory architecture for short read alignment in ReRAMs. *IEEE Computer Architecture Letters* 17, 2 (2018), 237–240. DOI : <https://doi.org/10.1109/LCA.2018.2854700>
- [30] F. Merrikh-Bayat, B. Hoskins, and D. B. Strukov. 2015. Phenomenological modeling of memristive devices. *Applied Physics A* 118 (2015), 779–786. DOI : <https://doi.org/10.1007/s00339-015-8993-7>
- [31] R. Berdan, C. Lim, A. Khiat, C. Papavasiliou, and T. Prodromakis. 2014. A memristor SPICE model accounting for volatile characteristics of practical ReRAM. *IEEE Electron Device Letters* 35, 1 (2014), 135–137. DOI : <https://doi.org/10.1109/LED.2013.2291158>
- [32] S. Jo, T. Chang, I. Ebong, B. B. Bhadviya, P. Mazumder, and W. Lu. 2010. Nanoscale memristor device as synapse in neuromorphic systems. *Nano Letters* 10, 4 (2010), 1397–1301. DOI : <https://doi.org/10.1021/nl904092h>

Received March 2019; revised September 2019; accepted November 2019